

Developing AI Agents with Agno

Assignment



Assignment: Developing a Customer Support Triage Agent for E-Commerce

Scenario

A growing e-commerce platform YNC receives thousands of customer queries per day related to refunds, shipping delays, product issues, and account problems. Currently, human agents manually categorize, prioritize, and respond to tickets—leading to delayed responses, misrouted tickets, and inconsistent service quality. The company wants to build an AI-powered Customer Support Triage Agent to automate this initial triage process, improve efficiency, and surface actionable insights for management.

Problem Statement

Create an AI Customer Support Triage Agent using Agno (formerly Phidata) that allows support teams to upload and analyze support logs (CSV/txt) and company policy documents (PDF). The agent should:

- Categorize tickets,
- Extract key information (e.g., sentiment, urgency, intent),
- Suggest replies,
- Support semantic search across past issues,
- And generate summaries of common problems for supervisors.

The solution should leverage document chunking, vector embeddings, and LLM-powered tools to deliver accurate triage and efficient support operations.

Objectives

- Design an AI Agent using Agno tailored for customer support triage.
- Ingest user-uploaded support logs and policy documents.
- Extract structured insights (intent, sentiment, urgency, topic).
- Generate draft responses and support routing decisions.

- Enable semantic search over past support queries.
- Build an interactive UI with a natural language chat interface for querying trends, complaints, and summaries.

Tasks to Be Implemented

1. Define the Agent

- Create a SupportTriageAgent class using Agno's Agent abstraction.
- Configure the agent with:
 - model: LLM (e.g., Gemini or OpenAI)
 - tools: for sentiment analysis, intent detection, response generation
 - embeddings: for support log retrieval
 - knowledge: for retaining historical complaint contexts

2. Data Upload & Preprocessing

- Implement file upload logic supporting:
 - .csv (customer support tickets)
 - .txt (internal notes or chat logs)
 - .pdf (support policies or documentation)
- Extract key fields: customer issue, timestamp, sentiment (if tagged), and response
- Use:
 - pandas for CSV parsing
 - pdfplumber or PyMuPDF for extracting PDF content

3. Chunking and Embedding

- Apply appropriate chunking strategies:
 - Split long support conversations into multi-turn chunks
 - Chunk PDF policies by section or topic
- Generate vector embeddings using sentence-transformers or Gemini API

4. Vector Database Integration

- Store chunked embeddings in a vector database (e.g., Pinecone)
- Enable semantic search for queries like:
 - “Show similar complaints about delivery delays”
 - “Find refund issues with negative sentiment in May 2024”

5. Tool Integration

- Integrate tools for:
 - Sentiment and urgency detection
 - Intent classification (e.g., refund, delivery, account issue)
 - Suggested response generation
 - Policy reference matching (retrieving relevant policy sections)
 - External lookup (optional): Wikipedia or DuckDuckGo
 - Calculator tool for refund eligibility checks (e.g., based on dates or thresholds)



6. Build Workflow

- Create a SupportTriageWorkflow using Agno's task orchestration
- SampleFlow:
Log Ingestion → Preprocessing → Chunking → Embedding → Intent/Sentiment
Extraction → Suggested Response → Query Handling

7. User Interface

- Use Streamlit to develop a clean UI with:
 - File upload interface (support logs, policies)
 - Summary dashboard (complaint types, top issues, resolution times)
 - Chat interface to query logs:
 - “Summarize refund-related complaints from last week”
 - “What are top 3 customer pain points this month?”
 - Response suggestion panel with editable drafts

Required Feature Suggestions

Multi-File Handling

- Upload and manage multiple support logs and policy docs
- Merge content for broader triage context

Knowledge Module

- Persistent store of past complaints and resolutions
- Reusable across sessions and projects

Hybrid Search

- Combine keyword search (e.g., "refund") with semantic search
- Return source reference in response

Agent Tools

- Auto-suggest draft responses for agents
- Highlight cases needing escalation
- Supervisor insights like issue spikes or SLA violations

Session State Management

- Track session history (queries, file uploads, responses)
- Use JSON or Pickle-based state saving

Note: Virtual Environment Setup

- Create Virtual Environment:
 - Navigate to project directory (e.g., D:\mod4\)
 - Run: `python -m venv venv`
 - Activate:
 - Windows: `venv\Scripts\activate`
 - macOS/Linux: `source venv/bin/activate`

- Install Dependencies:
 - Create requirements.txt with relevant packages (e.g., streamlit==1.35.0, sentence-transformers, pdfplumber, pandas)
 - Run: pip install -r requirements.txt

